Schnapsen for Strategy Game Engine
- Documentation -

Bina Philipp Christoph - 1173047

April 2026, TU Wien

Contents

1 Introduction **1**

2 Schnapsen Game **2**
2.1 Rules . . . 2
2.1.1 The deck of cards . . . 2
2.1.2 Goal of the game . . . 2
2.1.3 Preparation . . . 2
2.1.4 Gameplay . . . 3
2.1.5 The Talon . . . 3
2.1.6 Marriages . . . 4
2.1.7 Exchange the Trump Card . . . 4
2.1.8 Close the Talon . . . 4
2.1.9 The last Trick . . . 4
2.1.10 End of Round and "Bummerl" . . . 4
2.2 Adaptations to the Rules . . . 5
2.3 Running a Schnapsen Game in the Strategy Game Engine . . . 6

3 Schnapsen Agents **6**
3.1 Random Agent . . . 6
3.2 Alpha-Beta Agent . . . 6
3.3 MCTS Agent . . . 7

4 Agent Creation **7**
4.1 Setting up in Intellij IDEA . . . 7

1 Introduction

This project is an abstract implementation of the Austrian card game "Schnapsen" in the Strategy Game Engine from Lukas Grassauer [1], which has been forked and extended by Clemens Anzinger [2]. The engine makes it possible to let

implemented agents play against each other. Furthermore three basic agents have been implemented to compete against.

2 Schnapsen Game

`https://github.com/binapple/Schnapsen`

This section will explain the rules used for Schnapsen and implementations details that are useful for agent implementation.

2.1 Rules

This projects game is implemented based on the rules of the Viennese card producer Piatnik© [3]. Schnapsen is a two-player trick-taking card game. There are variations for three and four players, but this implementation focuses on the two-player version.

2.1.1 The deck of cards

The game is played with a set of 20 cards and four suits (Spades, Hearts, Diamonds, Clubs). Each suit features five cards in this ranking (highest to lowest): Ace, Ten, King, Queen, Jack. In Schnapsen the Ten is higher than the picture cards, which may be unusual for first time players. Each card has a certain score to itself: Ace (11), Ten (10), King (4), Queen (3), Jack (2). Generally, a card with a higher score beats the one with a lower score in the same suit, when determining the winner of a trick. **Disclaimer:** In Austria often times regional cards are used to play Schnapsen. In these cards the suits are "Laub" = Spades, "Herz" = Hearts, "Schellen" = Diamonds, "Eichel" = Clubs.

2.1.2 Goal of the game

The overall goal of one round of Schnapsen is to get 66 points (sum of the cards scores) in won tricks. There is another way to earn points by "marrying" a Queen and a King of the same suit, which will be described in detail later.

2.1.3 Preparation

At the start of a new round a starting player is decided, by shuffling the deck and having each player cut the pile. The higher card shown on the cut states who will start. The non-starting player is the dealer for the first round and deals 3 cards each and one card is put face-up as the trump card. The trump card decides which suit is "stronger" in this round, therefore the trump suit. Afterwards 2 more cards are dealt to each player for a total of 5 cards in each players hands. The rest of the cards are placed face-down as a draw-pile (called Talon).

2.1.4 Gameplay

The game starts with the starting player playing a card from their hand, therefore they are the leading player playing the leading card. They are free to choose any of their cards. The non-starting player now has to play one of their cards, therefore they have to follow, so they are the following player. One played card of each player create the so-called trick. As long as there is no active must follow suit or must take the trick rule (described later), the following player can freely decide to play a card on their own. The trick is then evaluated:

- If the following players card is of the same suit and has a higher score than the one played by the starting player the trick is won by the following player. If it is lower it is won by the leading player.
- If the cards suit does not match the leading one, the trick goes to the leading player.
  - **EXCEPTION:** If the card in the non-matching suit is of the same suit as the trump suit (same as the face-up trump card), the trick goes to the following player. So a leading card in trump suit can only be won by following with a higher card of the trump suit.

Either player who won the last trick is starting to lead in the next round, but before that every player gets to draw a new card from the draw-pile (Talon), starting with the player who won the last trick.

2.1.5 The Talon

If the Talon is empty (the face-up trump card is the last card in the pile), the games rules change: Now there is a must follow suit and a must take the trick rule. Reminder: For the rest of the round the trump suit is still the same as the now drawn trump card.

- **Must follow suit rule** states that the following player has to play a card with a matching suit if possible.
- **Must take the trick rule** states that the following player has to play a card in the same suit as the leading card and one that has a higher score. If not possible they have to use a card of the trump suit, if they have any.

Important notice: The must follow suit rule is stronger than the must take trick rule, but they are not exclusive. For example: The playing pile is empty, the trump suit is Hearts, the leading card is a Clubs card. Now the following player has to play in decreasing priority:

1. a higher Clubs card. If not possible than:
2. a lower Clubs card. Ff not possible than:
3. any heart card. If not possible than:
4. any remaining Diamonds or Spades card of their free choosing

2.1.6 Marriages

If player who gets to lead in the next trick has a pairing of a Queen and a King of the same suit, they can declare a Marriage. A Marriage is worth 20 points, a marriage of the trump suit is worth 40 points. To do so one would call "Twenty" and "Forty" respectively, show both cards and then lead the trick with either the Queen or the King just shown. The points for a marriage will only be counted to the players total if they take any trick in the round (or have taken already).

2.1.7 Exchange the Trump Card

When a player gets to lead the next trick (also when they are starting player), they may exchange the trump card if and only if they have the Jack of the trump suit. They swap the card with their Jack and get to have a card of a higher score. This card could by chance, even enable them to declare a "Forty" after the exchange.

2.1.8 Close the Talon

"Closing the talon" is an important trait of Schnapsen. When a player gets to lead the next trick they may close the Talon to enforce an earlier occurrence of the must follow suit and must take the trick rules in the ongoing round. To do this the player takes the up-facing trump card and places it face down on the drawing pile. This is a strategical chance to win the round early when the player has a strong hand of cards. One has to get 66 points with the remaining cards in their hand as the Talon or draw pile is now not available anymore. If they are not able to finish the round with 66 points, the non-closing player gets at least two points off the current "Bummerl" (more on these points in detail later).

2.1.9 The last Trick

If neither party was able to gain 66 points after the last trick, the player who won the last trick wins this round. This kind of victory is not possible for the player who has closed the Talon, they are obliged to reach 66 points.

2.1.10 End of Round and "Bummerl"

The game round ends when one player reaches 66 points in trick scores. When they are going to lead the next card, they can say "Thank you, enough" and the game ends. If they did not have enough points by then they have to give the points they would have got to the opposing player. To start a new round the player who has not been dealing this round shuffles the cards and deals the next round.

As a round of Schnapsen is played very fast and can include a huge factor of luck it is typical to play more than one round to determine a victor. A "Bummerl" was the name of kegs of beer in old times. Today it is used to track a series of rounds of Schnapsen. One Bummerl is seven points. Typically they are counted down, so each player starts at seven points. A winning player gets a differing amount of points depending on the score of the other players tricks. If the other party had zero tricks the winner gets three points. If they had at least one trick, the winner gets two and if they had more than 33 in scores, the winner gets one point. The scoring rules are different when the Talon was closed. Here the points depend on the non-closing players score **before** the Talon was closed. If it was higher than 33, one point for the closing player, if there was at least one trick two points and if there were none three for the closing player. If the closing player did not win the non-closing player gets at least two points and if they did not have any tricks prior to closing the non-closing player gets three points. So there is a certain risk to closing the Talon.

As stated before the points are counted down from seven, so the player who reaches zero first wins the "Bummerl". This is tracked by writing a dot to the losing player. It is common to play until one player loses a certain amount of "Bummerl". Did a player not win any points, therefore still have a "Bummerl" score of seven, they receive two dots.

2.2 Adaptations to the Rules

As the game is implemented as an abstract version of Schnapsen a few changes had to be made. Hidden information is a key part of this card game. Not knowing what your opponents cards or the cards in the draw pile are is therefore enforced by the code, by using "Hidden Card" placeholders. There are still some key aspects of the physical game that are not or differently implemented than the rules:

- The starting player for the very first round is always the first agent that was passed to the engine.
- Cards are automatically shuffled and passed to the players, that is true for the ones passed after each trick as well.
- As it makes no difference when the trump card is put face-up, the players get passed five cards alternately and then the next card is used as trump card.
- Players do not have to "remember" the tricks taken by them or the other party and their respective scores, as this is technically "open" information.
- Players are only presented "legal" cards to play as an action, so the are not able to break the must follow suit and must take the trick rule.
- Players do not have to say "Thank you, enough", the game tracks all points and stops the round when one party reaches 66 in scores.

2.3 Running a Schnapsen Game in the Strategy Game Engine

The Strategy Game Engine (SGE) provides an interface for the implementation of games. Schnapsen follows the provided manual that can be found at [1]. The exception is the "-b" option which is used to control the starting seed of the game and the amount of Bummerl a player needs to have to lose. The starting seed controls all occurences of randomness in the game like how the card pile is shuffled. The schematic is the following -b "<amount of Bummerl >;<starting seed>", for example -b "3;2026" starts the game with the seed 2026 and it ends when one player has reached three Bummerl. When using this option one can also only declare a number of Bummerl to be played, so -b "2" starts the game with a random seed and it ends when one player reaches two Bummerl. A complete command for running a game of Schnapsen could look like this:

```
java -jar .\sge-1.0.4-dq-exe.jar match
-c 25
-b "3;2026"
.\Schnapsen-1.0.jar
.\Schnapsen_Agents_Alpha_Beta.jar
.\Schnapsen_Agents_Random.jar
```

This is the same structure that is used to start any game in the SGE.

3 Schnapsen Agents

`https://github.com/binapple/Schnapsen_Agents`

The Agents provided should give examples on how to work with the Schnapsen Game. The biggest difference in working with the Schnapsen game compared to perfect information games is that there is hidden information. The cards of the opposing player and the face-down cards in the drawing pile are replaced with "Hidden Card" placeholders. To tackle this problem the MCTS-Agent and the Alpha-Beta-Agent have a method to determinize the game, called generateMissingInformation().

3.1 Random Agent

The random agent picks one of the available actions at random.

3.2 Alpha-Beta Agent

The alpha-beta agent creates one random perfect Information game of Schnapsen with the mentioned generateMissingInformation(). On this determinization an alpha-beta algorithm with pruning based on pseudo-code from Russell et al. [4] is run. The depth of the search tree is restricted to the current round, end of the game or if there is not enough computational time left.

3.3 MCTS Agent

The Monte Carlo Tree Search (MCTS) Agent is also determinizing the game before running an Upper Confidence Bounds for Trees (UCT) algorithm based on the one described by Browne et al. [5]. The algorithms depth restrictions are the same as in the alpha-beta agent.

4 Agent Creation

To create a working Schnapsen agent for the SGE it is necessary implement the GameAgent¡Schnapsen, SchnapsenAction¿ interface. It is helpful to extend the AbstractGameAgent¡Schnapsen, SchnapsenAction¿ as well to get access to control methods like shouldStopComputation(). An agent has to have a constructor that contains only the Logger of the SGE, which will be called automatically by the engine when creating the agent. (More details in [1])

The agent will receive a version of Schnapsen tailored to the agents point of view (with "Hidden Card" placeholders for the hidden information) through the computeNextAction() method call by the engine. When trying to play the game with these placeholder cards, exceptions will be thrown, so it is necessary to somehow fill in this missing information. Be aware that also the seed for the random object is not the same as the one of the "original" board, so drawing/passing and shuffling cards will have different outcomes in the game version that is passed to the players, which is one of the reasons that keeping the search tree in the current round makes sense. As mentioned above, the generateMissingInformation() method used by the alpha-beta and MCTS agent shows one possible solution to create a playable state of the game.

4.1 Setting up in Intellij IDEA

To start developing your own Schnapsen agent in Intellij IDEA you click on **File** > **New** > **Project**.

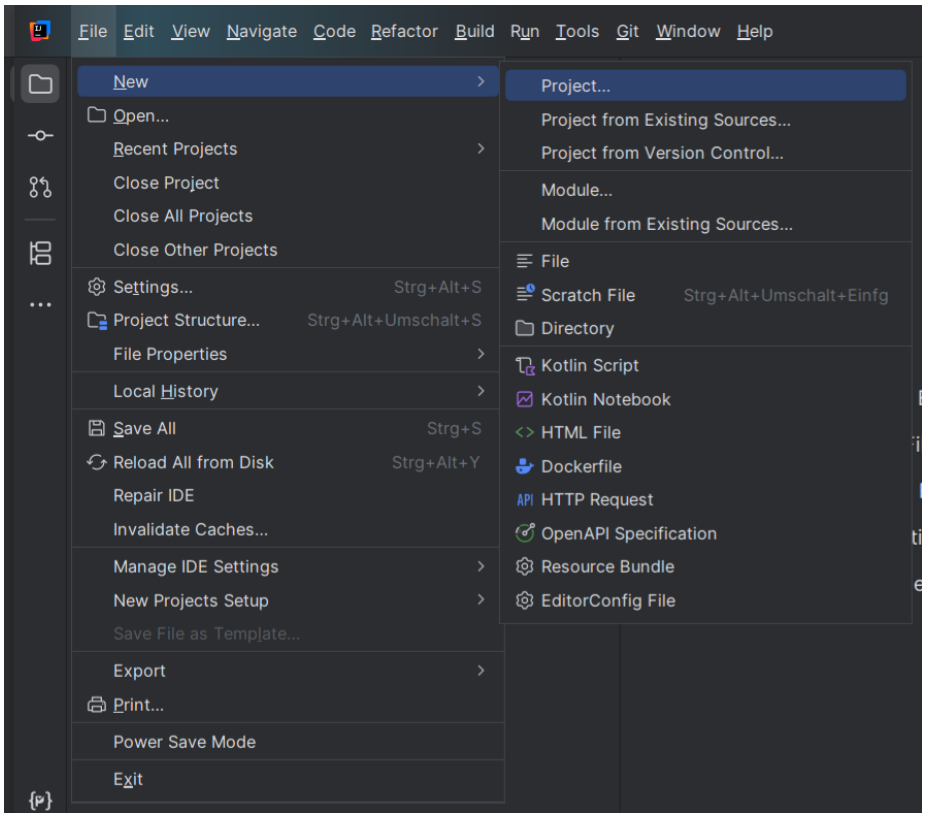

Figure 1: Create a new project

In the "New Project" window you will have to choose **Java** as Project type and give you agent project a name. Choose Build system **Gradle** with **Kotlin** as DSL.

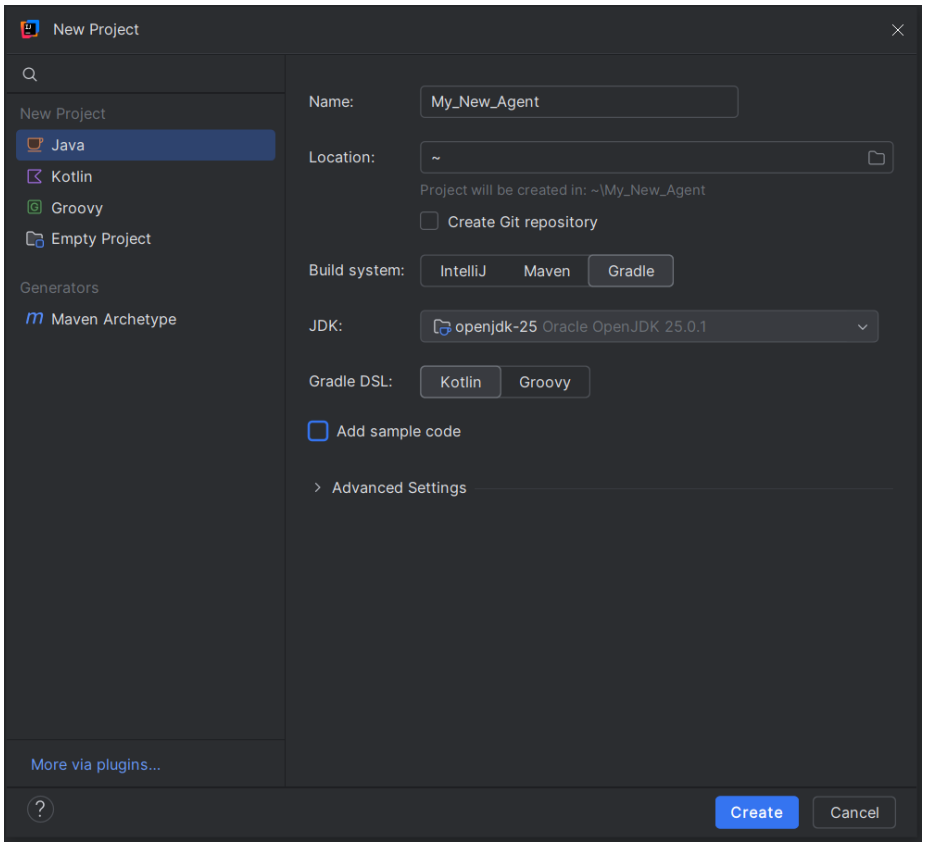

Figure 2: New Project settings

In this new project you can now add a folder for the third party software or external libraries in our case the sge-1.0.4-dq-exe.jar and the Schnapsen.jar. Add a new package and a Java class according to your agents name in the src/main/java folder. Your projects file structure could look like this:

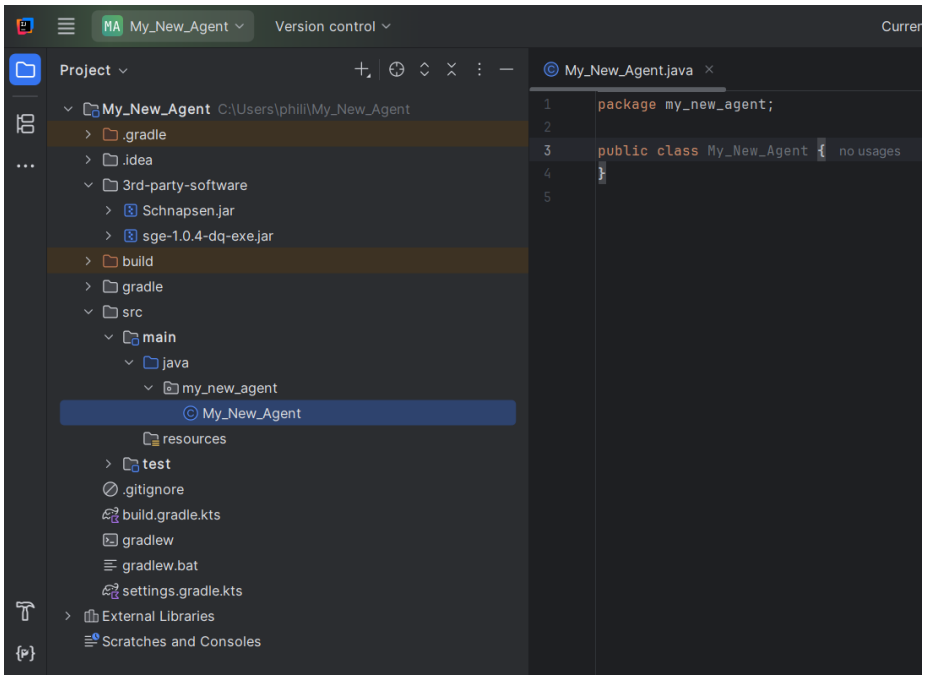

Figure 3: Possible File structure

In your project you should see a "build.gradle.kts" file. You can edit or replace it with the following code. Make sure to use the names of your folders, package and Java class.

```
plugins {
    id("java")
}

group = "at.tuwien"
version = "1.0-SNAPSHOT"

repositories {
    mavenCentral()
}

dependencies {
    testImplementation(platform("org.junit:junit-bom:5.10.0"))
    testImplementation("org.junit.jupiter:junit-jupiter")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
    implementation(files("$projectDir/3rd-party-software/sge-1.0.4-dq-exe.jar",
        "$projectDir/3rd-party-software/Schnapsen.jar"))
}
```

```
tasks.test {
    useJUnitPlatform()
}

var agent = "My_New_Agent"
var package_path = "my_new_agent"


tasks.jar {
    manifest {
        attributes(
            "Sge-Type" to "agent",
            "Agent-Class" to package_path + "." + agent,
            "Agent-Name" to agent,
        )
    }
}
```

Now you can resync the project using **Build** > **Sync Gradle Project**.

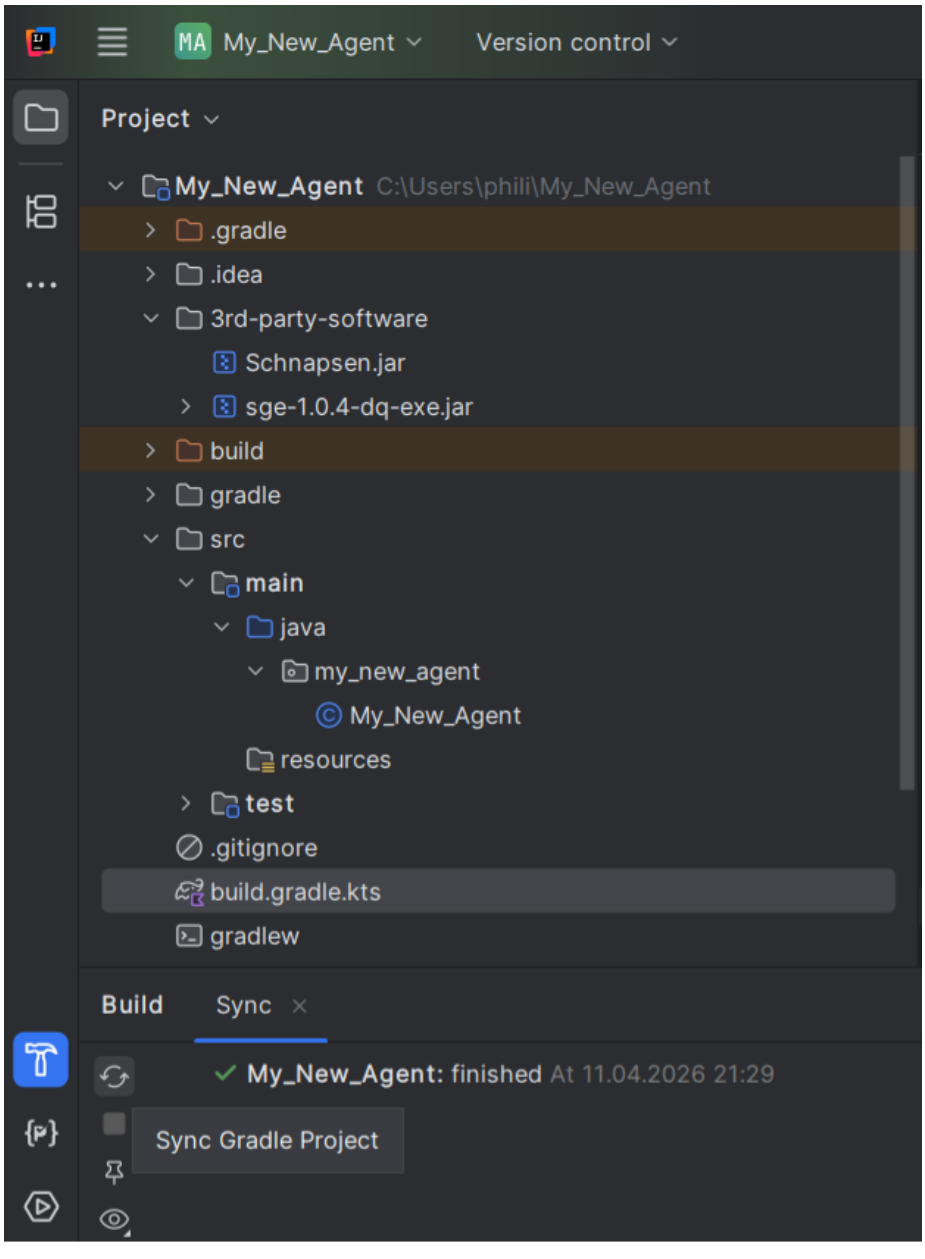

Figure 4: Gradle resync

Building your project can be done with **./gradlew build**. You find the agents jar file in the **build/libs** folder.

You can test if everything works by adapting the following code building

your agent and giving it a test run against the example agents.

```
public class My_New_Agent  extends
    AbstractGameAgent<Schnapsen, SchnapsenAction> implements
    GameAgent<Schnapsen, SchnapsenAction> {

    public My_New_Agent(Logger log) {
        super(log);
    }

    /**
     * This method will be called by the engine everytime
        the agent has its turn.
     * In this method the agent looks at all the possible
        actions to take and picks a random one of them to
        return.
     * @param schnapsen the games state as given by the
        engine (may include hidden information)
     * @param computationTime the maximum available time
        the agent is allowed to take to think about its
        next action
     * @param timeUnit the unit in which the
        computationTime is measured
     * @return a random SchnapsenAction chosen from the
        agents available ones
     */
    @Override
    public SchnapsenAction computeNextAction(Schnapsen
        schnapsen, long computationTime, TimeUnit timeUnit) {
        Set<SchnapsenAction> actions =
            schnapsen.getPossibleActions();
        Object[] actionArray = actions.toArray();
        int actionCount = actionArray.length;
        int pickedActionId = new
            Random().nextInt(actionCount);
        return (SchnapsenAction)
            actionArray[pickedActionId];
    }

}
```

References

[1] L. Grassauer, "Entze/Strategy-Game-Engine," Mar. 2025, original-date: 2021-09-13T12:04:53Z. [Online]. Available: https://github.com/Entze/Strategy-Game-Engine

[2] canzinger, "canzinger/Strategy-Game-Engine-Disqualification-Extension," Nov. 2025, original-date: 2023-02-22T09:52:43Z. [Online]. Available: https://github.com/canzinger/Strategy-Game-Engine-Disqualification-Extension

[3] "Piatnik - Karten-Spielregel." [Online]. Available: https://www.piatnik.com/karten-spielregel

[4] S. J. Russell and P. Norvig, *Artificial intelligence: a modern approach*, ser. Prentice Hall series in artificial intelligence. Englewood Cliffs, N.J: Prentice Hall, 1995.

[5] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, "A Survey of Monte Carlo Tree Search Methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, Mar. 2012. [Online]. Available: https://ieeexplore.ieee.org/document/6145622/